

CANNBot支持生成单指令多线程算子

作者：管宏伟

时间：2026年06月

目录

CONTENTS

Part 01 背景

Part 02 实操演示

Part 03 工程概览

Part 04 SIMT领域知识设计

Part 05 Agent Team与Skill策略设计

Part 06 工程交付体系设计

Part 0: 前置知识—什么是SIMT编程

1. 什么是单指令多线程算子?

SIMT (Single Instruction, Multiple Threads, 单指令多线程) 是昇腾 950 提供的并行多线程编程能力。核心思想: 以**线程**为并行单位, 32 个线程组成一个 **Warp**, Warp 内所有线程在同一时刻执行同一条指令, 但每个线程拥有独立的程序计数器 (PC) 和寄存器, 可走不同分支路径。

3. 核心差异: SIMT 对比 SIMD

SIMD: 单指令多数据

基于向量单元, 按照block粒度处理数据, 用一条指令同时对**多个数据元素**执行相同的操作, 从而实现数据级并行, 提升处理效率。

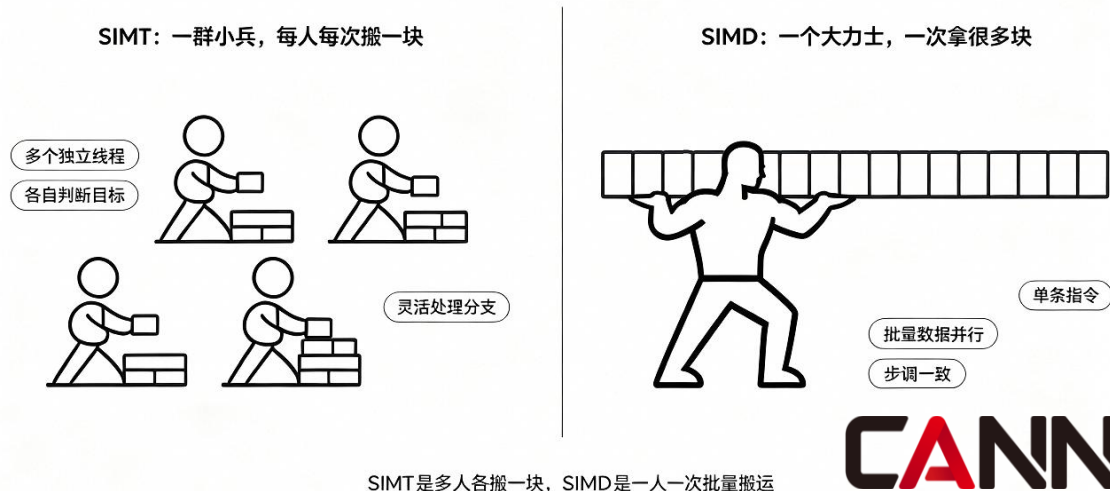
需要程序员关注UB大小, 且数据要排布整齐, 为了高性能还需要开pingpong处理。

所以, SIMT在编程易用性上, 复杂分支场景上比较适合, 且性能优于SIMD

2. SIMT关键特性

- 线程级抽象: 程序员以"线程"为单位编写代码, 无需关心向量长度, 硬件自动调度
- Warp访存合并: 硬件自动检测同一 Warp 线程的全局内存访问地址, 连续对齐访问合并为少量内存事务
- 分支发散 (Branch Divergence): 当 Warp 内线程走不同分支时, 硬件会串行执行各分支路径

SIMT 与 SIMD:



Part 1：背景

传统算子开发方式：

痛点	传统方式
开发门槛高	需要深入理解昇腾硬件架构、Tiling 切分策略、SIMT 线程模型
开发周期长	一个算子从设计到交付通常需要 数周
质量难保证	竞品对齐、精度一致性、代码规范等靠人工审查，容易遗漏

借助AI开发方式：

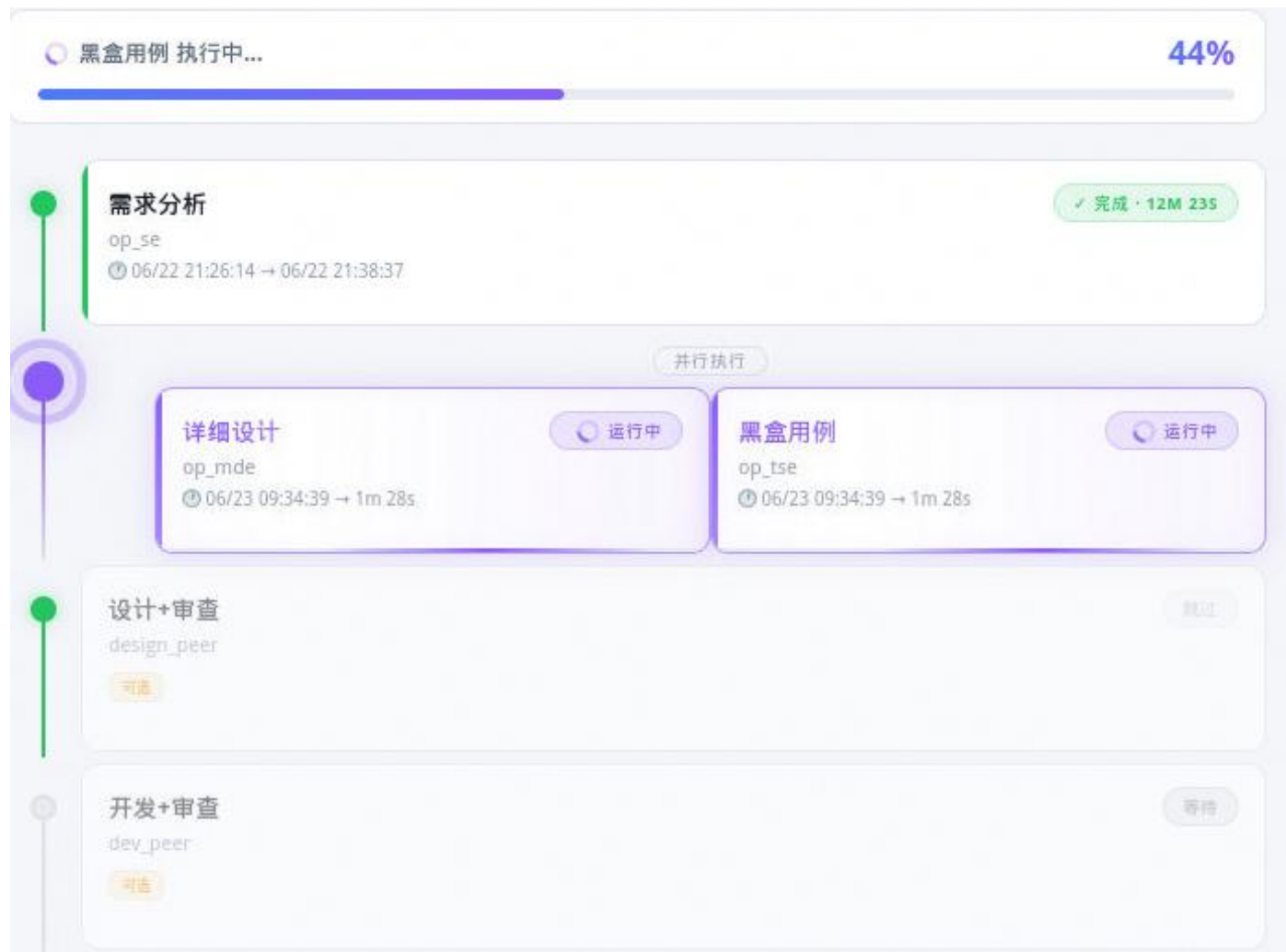


从 纯手工——>简单作坊——>精细作坊——>半自动作坊——>自动化AI工厂 的演进之路



Part 2: 实操演示

帮我开发一个trilu算子，开启可视化



Part 3：工程概览

工程目标：让LLM学会SIMT编程，构建三层纵深防御体系

核心风险：用LLM生成代码面临的三大挑战

过程难点

知识缺失
缺乏特定领域专业知识

幻觉严重
易产生不符合事实内容

过程不可控
生成过程难追溯验证

解决方案

知识编码
让LLM学会SIMT编程

团队与Skill
压缩幻觉空间

交付体系
兜底幻觉逃逸

设计思路

源头：知识注入

输入端：压缩空间

输出端：拦截逃逸

Part 4: SIMT领域知识设计：教LLM学会SIMT算子编码



编程模型知识编码

将昇腾SIMT独特的线程模型、内存空间、函数调用等刚性规则，编码为“不遵守就会编译失败”的底线规则。

模型	skill
线程模型	以线程为并行单位，32 线程=1 warp；层次 gridDim→blockDim→threadIdx，线程数编译期常量，默认 1024、上限 2048。
内存空间	UB 256KB = 静态+动态+8KB 预留+DCache，DCache≥32KB，可用约 216KB；GM 可直接读写。
函数调用层级	核函数→ simt_vf (经 asc_vf_call 启动)→ simt_callee ，VF 内只能调 callee 与 constexpr。
寄存器约束	每线程寄存器随线程数反比（256→127...2048→16），超出栈溢出。
切分策略	核数两步法：物理核均分→反推 needCore；PER_CORE_MIN≥1024 且对齐 32。
线程→数据映射	Grid-Stride（独立逐元素）/ Warp 分片+shfl（规约）/ Warp 级操作（排序）。

Part 4: SIMT领域知识设计：教LLM学会SIMT算子编码



算法模式策略库

将复杂算子归纳为逐元素并行、分段归约等六类核心算法模式，并为每类模式封装设计决策树与实现策略。

模式	代表算子	核心策略	关键设计决策点
逐元素并行	ReLU, Add, Mul, Cast	一维 grid + 步长循环 + float4 向量化 + 128B 合并访问	向量化宽度选择、grid 维度决策
分段归约	ReduceSum, Softmax, LayerNorm	沿 reduce 轴 tiling + 树形归约 + warp shuffle + atomicAdd 跨 block 收尾	归约趟数设计 (Softmax 三趟防溢出)、warp 级 vs block 级归约选择
滑窗	MovingAverage, RollingMax	增量更新 $O(W) \rightarrow O(1)$ + 共享内存预取 ($W \geq 5$) + 多通道展开	滑窗宽度阈值判断 ($W < 5$ 直接全局读 vs $W \geq 5$ 共享内存)、通道维度 grid 展开
前缀积累	CumSum, CumProd	Blelloch 上扫+下扫 + 跨 block 二次 scan	block 间前缀传播策略、bank conflict 规避
任意轴 Broadcast	BroadcastAdd, Where	stride 查表坐标映射 + 连续维度向量化 + 被广播值寄存器复用	广播维度检测、反向 broadcast 的 atomicAdd 归约
索引访问	Embedding, ScatterAdd, IndexSelect	Gather 流水线掩盖延迟 + Scatter 冲突消除策略矩阵	冲突率判据 ：索引唯一→直接 atomic；中等重复→warp 检测；高重复→排序法；小规模→共享内存 batch

Part 5: Agent Teams与Skill策略框架设计—Harness设计



Part 6: 工程交付体系设计



总结：三层纵深防御体系



知识编码（源头）

解决“不知道”的问题
将SIMT领域知识编码为Skill，让LLM学会原本不具备的专业编程模型。



Agent Teams与Skill策略框架（压缩）

解决“不可靠”的问题
通过多Agent分治、Gene式策略注入、刚柔混合、异构博弈压缩幻觉空间。



交付体系（兜底）

解决“不可信”的问题
四层拦截、角色隔离、断点续传、标准化流水线确保代码可信可用。

- 1. 编程模型知识——
- 2. 算法模式知识——

幻觉根因	设计决策
注意力失焦	—> 3.1 Agent Teams 分治设计
职责混淆	—> 3.2 Agent与Skill分离设计
	3.3 Skill 策略框架设计
	3.4 知识按需加载设计
知识缺陷	—> 3.5 示例设计
约束缺失	—> 3.6 刚柔混合执行设计
	3.7 结构化输出设计
多模型共犯概率低	—> 3.8 异构博弈审查设计

残余幻觉逃逸路径	拦截机制
结构性/接口错误	—> 4.1 四层拦截保障设计
角色越界/流程失控	—> 4.2 角色隔离与交付件驱动设计
中断后不可恢复	—> 4.3 断点续传与重试设计
流程无纪律	—> 4.4 八阶段流水线SOP设计

CANNBench 评测总榜

💡 simt模式：端到端生成12个算子，精度通过率100%。

4. Operator Details / 算子明细

4.1 Level 1 — 基础算子

Table 4. Level 1 Operator Results

#	OPERATOR	CATEGORY	LEVEL	CASES	PASSED	PASS RATE	AVG PRECISION	AVG SPEEDUP	TOTAL SCORE
1	ForeachAddcdvScalar	FusedComposite	L1	20	20	100%	100%	0.93x	72
2	SwiGlu	Elementwise	L1	20	20	100%	100%	9.62x	79

4.2 Level 2 — 中级算子

Table 5. Level 2 Operator Results

#	OPERATOR	CATEGORY	LEVEL	CASES	PASSED	PASS RATE	AVG PRECISION	AVG SPEEDUP	TOTAL SCORE
1	ApplyRotaryPosEmb	FusedComposite	L2	20	20	100%	100%	2.33x	74
2	Cummin	Reduction	L2	20	20	100%	100%	6.24x	92
3	Gather	IndexGather	L2	20	20	100%	100%	62.48x	91
4	RmsNorm	Normalization	L2	20	20	100%	100%	0.52x	63
5	Scatter	ScatterUpdate	L2	20	20	100%	100%	3.87x	77
6	Softmax	Normalization	L2	20	20	100%	100%	0.64x	67

4.3 Level 3 — 高级算子

Table 6. Level 3 Operator Results

#	OPERATOR	CATEGORY	LEVEL	CASES	PASSED	PASS RATE	AVG PRECISION	AVG SPEEDUP	TOTAL SCORE
1	AddRmsNormDynamicQuant	FusedComposite	L3	20	20	100%	100%	0.44x	63
2	DequantSwigluQuant	FusedComposite	L3	20	20	100%	100%	0.87x	67
3	TopK	SortSelect	L3	20	20	100%	100%	0.01x	50
4	Transpose	LayoutTransform	L3	20	20	100%	100%	0.98x	70

CANN-Bench 认证



本评测报告由 CANN-Bench 评测框架自动生成，评测对象为 DeepSeek V4 Pro (CANNBot) 生成的 Ascend C 算子代码。
评测数据来源于 CANN-Bench tasks/ 标准化题库，评分严格按照 CANN-Bench V0.1.0 评分规范执行。



现在，开始您的CANNBot之旅~ 让高效开发算子触手可及 ^_^

<https://gitcode.com/cann/cannbot-skills>

CANNBot代码仓库地址
欢迎star、fork、download、PR



邀请您扫码加入 “CANNBot Skills开源交流群”

群聊：CANNBot Skills开源交流
2群



该二维码7天内(7月1日前)有效，重新进入将更新

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力生态

社区使命：使能开发者基于CANN社区自主创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Build an open, easy-to-use, and technology-leading AI computing power ecosystem

Mission: Enable developers to innovate independently based on the CANN community, and build a deeply rooted, cross-industry collaborative, shared and win-win CANN ecosystem



上CANN社区获取干货



关注CANN公众号获取资讯